Module 7: Portfolio Milestone

CSC450-1 Programming III, Colorado State University – Global Campus

Professor H. Pensado

Portfolio Project: Part 1

Hugo Martinez

March 27, 2026

Concurrency – Counter Application

Concurrency allows a program to perform multiple operations at the same time, often by dividing work across separate threads. While this concept is commonly used in more complex systems, this application provides a straightforward example of how threads can be created, managed, and controlled.

In this program, two threads are used to simulate a counting process. The first thread counts upward from 0 to 20, while the second thread counts downward from 20 to 0. Although both tasks are handled using threads, they are executed in a controlled sequence rather than simultaneously. This is achieved through thread synchronization, ensuring that the second thread only begins after the first thread has completed. This approach highlights not only how concurrency works, but also how it can be managed to produce predictable and organized results.

In addition to building the application, this project also takes a closer look at some of the key considerations that come with using concurrency. While threads can improve how a program performs, they can also introduce overhead and complexity if not managed properly. This includes performance trade-offs, as well as potential security concerns related to how strings are handled and how data types are used.

Pseudocode

The following pseudocode outlines the overall logic and flow of the application. In particular, it highlights how the program separates the counting tasks into two distinct functions and uses thread control to ensure they execute in a specific sequence. Although both operations are handled using threads, the design intentionally prevents them from running at the same time by requiring the first thread to complete before the second begins. This controlled approach helps demonstrate not just how threads are created, but how they can be managed to produce predictable results.

```
BEGIN PROGRAM


IMPORT necessary libraries for threading and timing


DEFINE FUNCTION countUp

    FOR counter FROM 0 TO 20

        DISPLAY "Counting Up: " + counter

        PAUSE for a short duration (to simulate work and improve readability)

    END FOR

END FUNCTION


DEFINE FUNCTION countDown
```

```
    FOR counter FROM 20 DOWN TO 0
        DISPLAY "Counting Down: " + counter
        PAUSE for a short duration
    END FOR
END FUNCTION

MAIN FUNCTION

    DISPLAY "Starting Thread 1: Count Up"

    CREATE Thread1 TO execute countUp function

    START Thread1

    WAIT for Thread1 to complete execution

    DISPLAY "Thread 1 complete. Starting Thread 2: Count Down"

    CREATE Thread2 TO execute countDown function

    START Thread2

    WAIT for Thread2 to complete execution

    DISPLAY "Program complete"

END MAIN

END PROGRAM
```

C++ Implementation

The following C++ implementation brings that logic into a working program. While the program uses two separate threads to perform the counting tasks, it intentionally enforces a sequential flow by using the join() function. This ensures that the first thread completes its

execution before the second thread begins, preventing any overlap and maintaining a clear and predictable output. Additionally, the program separates each counting task into its own function, which not only improves readability but also aligns with how threads are typically assigned specific units of work. This design makes the program easier to follow and sets a consistent structure that can be replicated in other languages, such as Java.

```
#include <iostream>
#include <thread>
#include <chrono>

using namespace std;

// Function for Thread 1: Count up from 0 to 20
void countUp() {
    for (int i = 0; i <= 20; i++) {
        cout << "Counting Up: " << i << endl;
        this_thread::sleep_for(chrono::milliseconds(200)); // Pause for readability
    }
}

// Function for Thread 2: Count down from 20 to 0
void countDown() {
    for (int i = 20; i >= 0; i--) {
        cout << "Counting Down: " << i << endl;
        this_thread::sleep_for(chrono::milliseconds(200)); // Pause for readability
    }
}

int main() {
    cout << "Starting Thread 1: Count Up" << endl;

    // Create and start Thread 1
    thread thread1(countUp);
    thread1.join(); // Wait for Thread 1 to finish

    cout << "Thread 1 complete. Starting Thread 2: Count Down" << endl;
```

```
    // Create and start Thread 2
    thread thread2(countDown);
    thread2.join(); // Wait for Thread 2 to finish

    cout << "Program complete" << endl;

    return 0;
}
```

Program Execution and Results

The following output demonstrates the execution of the program, showing how each thread runs in sequence. As shown, the first thread completes its count from 0 to 20 before the second thread begins counting down from 20 to 0. This confirms that thread synchronization was successfully implemented using the join() function.

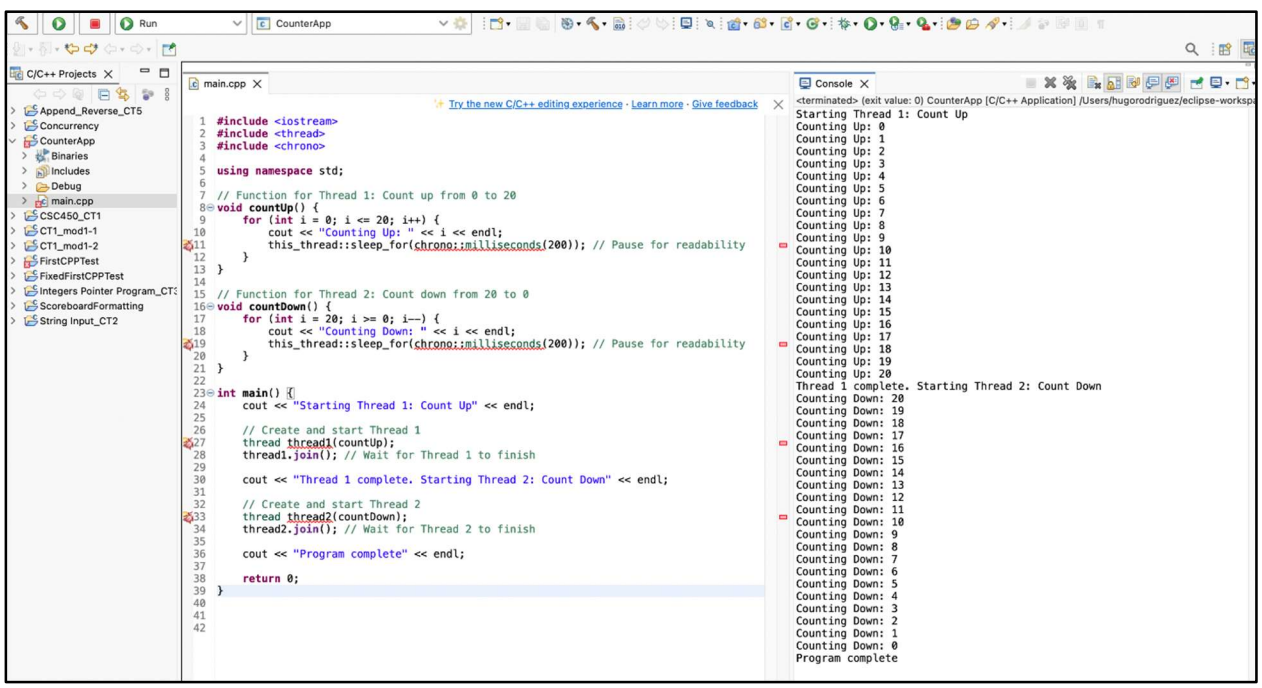


Figure 1. Program execution performing Count Up and Count Down instruction.

Program Analysis

Beyond just making the program work, it is important to understand how threading can impact performance, as well as how certain design choices, such as string handling and data type selection, can introduce potential vulnerabilities. The following sections take a closer look at these areas.

1. **Performance Issues with Concurrency**

One of the primary factors when working with concurrency is its impact on performance. While threads are often used to improve efficiency by allowing tasks to run in parallel, they also introduce overhead. Creating and managing threads requires additional system resources, and context switching between threads can add further processing cost.

In this program, the use of concurrency does not provide a performance benefit because the threads are intentionally executed in sequence using the join() function. This means that while threads are used, the program does not take advantage of true parallel execution. This design choice is intentional, as it ensures predictable and organized output.

2. **Vulnerabilities with Strings**

Another consideration involves how strings are used and handled. In C++, string handling can introduce vulnerabilities if not managed carefully, particularly when using low-level character arrays. Issues such as buffer overflows can occur when input exceeds the allocated memory space, potentially leading to unpredictable behavior or security risks.

The program uses standard output streams (cout) to display string messages, which reduces the risk of these types of vulnerabilities since no direct user input or manual memory management is involved. However, in more complex applications where user input is processed, it becomes critical to validate and properly handle string data.

Compared to other languages such as Java, C++ provides more flexibility in how strings are handled, but this flexibility comes with increased responsibility. Developers must ensure that proper safeguards are in place to prevent misuse or unintended behavior.

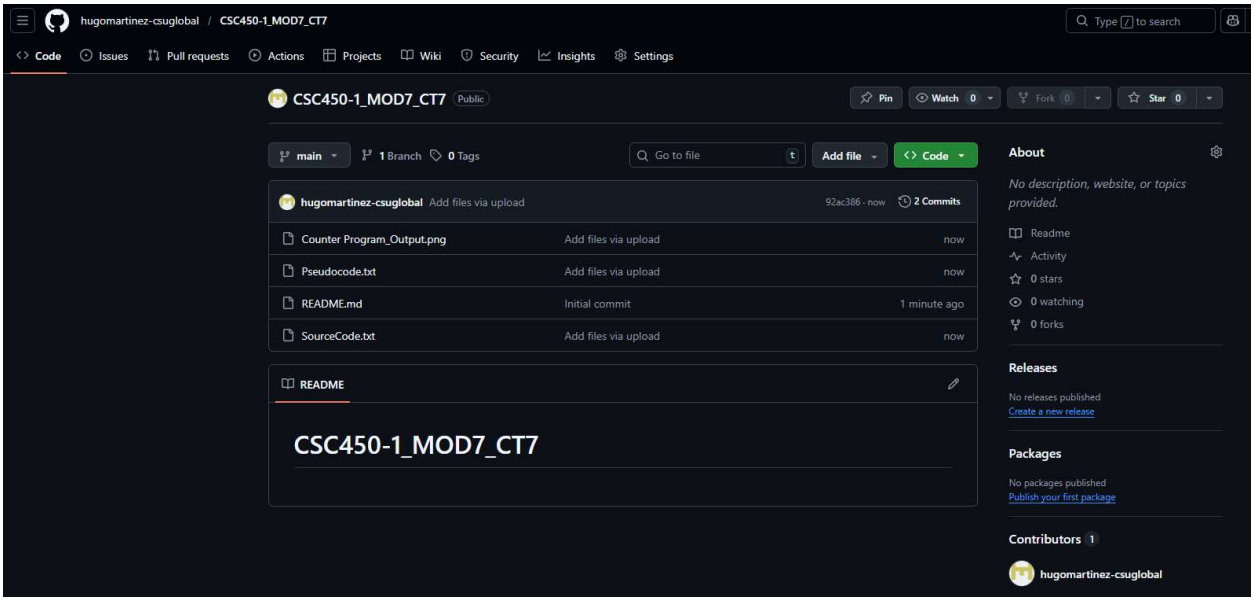
3. **Security of Data Types**

The choice of data types is another important factor when considering program reliability and security. In this program, integer variables are used to control the counting operations. While this works for the given range, it is important to recognize that improper use of data types can lead to issues such as overflow or unintended data manipulation.

In larger applications, selecting the correct data type helps ensure that values are stored and processed safely. In this case, the use of simple integer values within a controlled range minimizes risk. However, this still demonstrates the importance of choosing appropriate data types based on the requirements of the program. Languages like Java often provide additional safeguards and abstractions that reduce some of these risks, whereas C++ gives developers more direct control, which must be managed carefully.

GitHub Repository

The submission details can be found at the following GitHub Repository link: https://github.com/hugomartinez-csuglobal/CSC450-1_MOD7_CT7



Conclusion

By separating the counting tasks into two threads and controlling their execution using synchronization, the program demonstrates how concurrency can be implemented in a structured and predictable way. While the application itself is straightforward, it highlights the importance of understanding how threads operate and how their behavior can be managed effectively.

This project also emphasized the broader considerations that come with using concurrency. Performance trade-offs, along with potential vulnerabilities related to string handling and data types, show that writing functional code is only one part of the process. Equally important is understanding how design decisions can impact the reliability, efficiency, and security of an application.